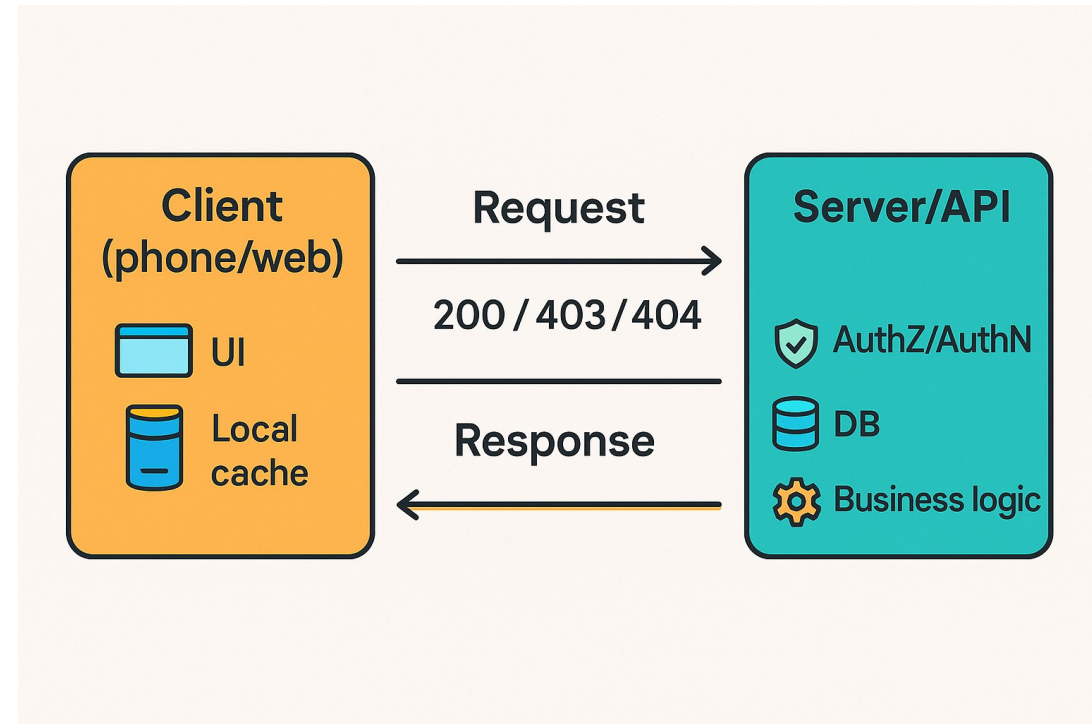


Mobile and Embedded Computing

Lecture 5. Server vs Client-Side code execution, Serverless vs VPS, http libraries, Firebase, Code generation

Server vs Client-Side Code Execution

- **Client-side** runs on user devices (apps, browsers). Great for rendering UI, local state, caching, basic input checks.
- **Server-side** runs on managed machines or platforms you control. Great for secrets, durable storage, heavy compute, and enforcing authorization.
- HTTP request/response cycle: client sends request → server executes → returns status + body.



Security responsibility



Treat all client input as untrusted; validate on the server (syntax + semantics). Client checks are helpful UX, not sufficient security.



Typical server-side validations: content type, ranges, cross-field rules, auth checks.



Consequences of skipping server validation: injection, corrupted state, data leaks.

Serverless vs VPS

- Functions-as-a-Service: deploy function code; platform handles provisioning, scaling, and patching.
- **Billing:** pay per invocation and resource time (no charges while idle). Examples: AWS Lambda, Cloud Functions/Cloud Run functions.

Serverless benefits & downsides

Benefits

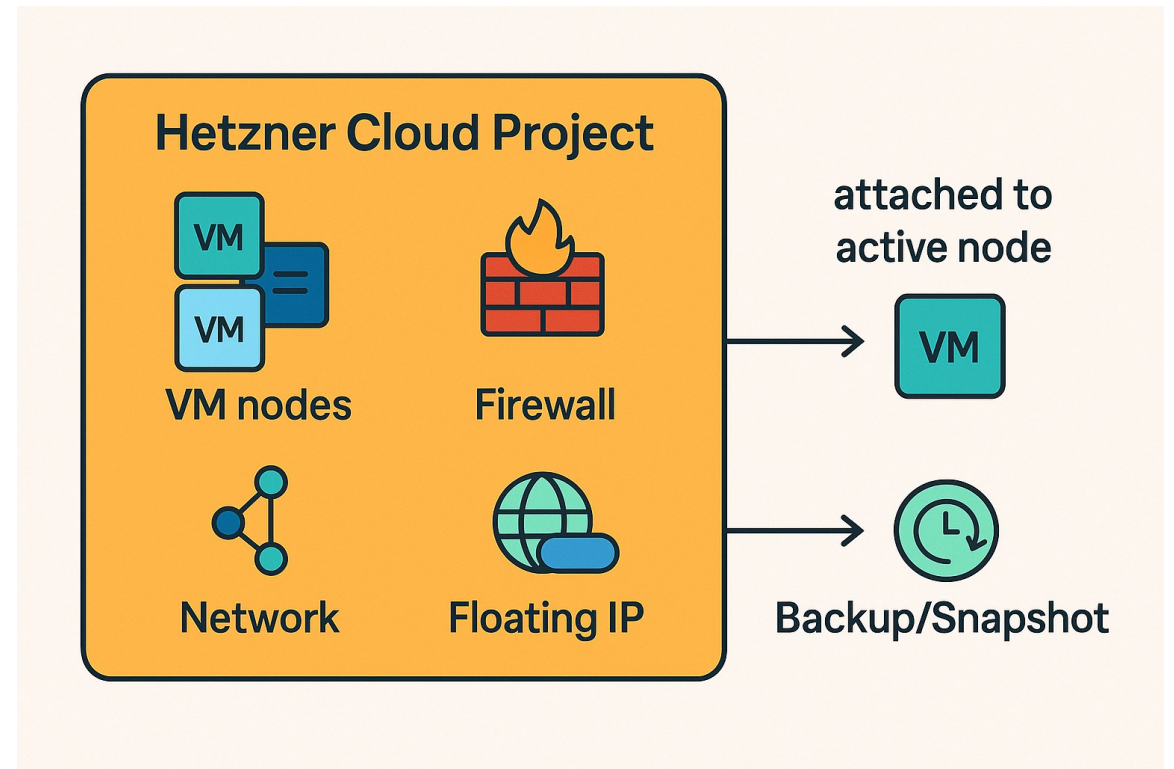
- Automatic scaling to zero/peaks; ops surface is small.
- Fine-grained cost alignment to usage.

Downsides

- **Cold starts** add latency; mitigations include keeping **min instances** / **provisioned concurrency**.
- Stateless runtime and execution time limits; limited control of networking/runtime.
- Vendor-specific limits and configuration models.

VPS (e.g. Hetzner Cloud)

- You rent a VM: pick CPU/RAM/disk, install OS, configure everything (networking, security, runtime).
- Hetzner Cloud supports **Firewalls, Floating IPs** (moveable IP for HA), **Backups/Snapshots**.
- Billing is hourly/monthly; you pay even when idle. (See Hetzner pricing overview).



Serverless vs VPS

Scaling: Serverless auto; VPS = you design autoscale (or scale manually).

Latency: Serverless may have cold starts; VPS is warm but you manage capacity.

Control: Serverless sandboxed; VPS full root control (kernels, agents).

Security posture: Serverless shares more infra but is managed; VPS requires you to patch, firewall, rotate keys.

Cost: Serverless shines for spiky/low traffic; VPS often wins for steady loads or special workloads.

Coding tips — Serverless (Lambda, Cloud Functions)

Move	Warm-path init: Move SDK clients/DB pools to global scope to allow reuse across invocations. Google Cloud+1
Start	Cold start hygiene: Trim dependencies; lazy-load rarely used modules; consider min instances/provisioned concurrency for latency-sensitive endpoints. Google Cloud Documentation+2Firebase+2
Prefer	Connections: Prefer HTTP keep-alive; reuse clients; understand concurrency scaling (each concurrent request → separate runtime slot). AWS Documentation+1
Time	Time limits & idempotency: Design handlers to be idempotent where possible to allow safe retries.

Coding tips — VPS

Default	Security first: Default-deny Firewalls, explicit allow rules; keep OS patched; use SSH keys. Hetzner Docs
Use	Resilience: Use Snapshots/Backups before upgrades; rehearse restores. Hetzner Docs
Put	Failover: Put a Floating IP in front; reassign on node failure. Hetzner Docs
Monitor	Ops: Monitor CPU/RAM/disk IO; automate with Ansible/Terraform (Hetzner APIs are available).

Flutter HTTP Libraries



Options

package:http: simple, composable; RetryClient helper available.

dio: rich features (interceptors, cancel tokens, form/multipart, transformers).

Chopper: declarative API client generator (via code gen).

retrofit.dart: annotations + generator on top of Dio.

dart:io HttpClient: low-level control (TLS, sockets) when you need it.

Practical tips



Set **timeouts** (connect/read) explicitly.



Use Dio **interceptors** for auth headers, logging, unified error mapping.



Cancel in-flight requests on screen dispose (Dio CancelToken).



Prefer codegen clients (Chopper/retrofit) for larger APIs to keep call sites typed and consistent.

Retry Mechanisms

HTTP **idempotent** methods (GET, HEAD, PUT, DELETE, OPTIONS) are defined to be safe to repeat; beware POST/PATCH unless you add idempotency keys.

Respect server hints: **Retry-After** header gives wait time (e.g., after 429/503).

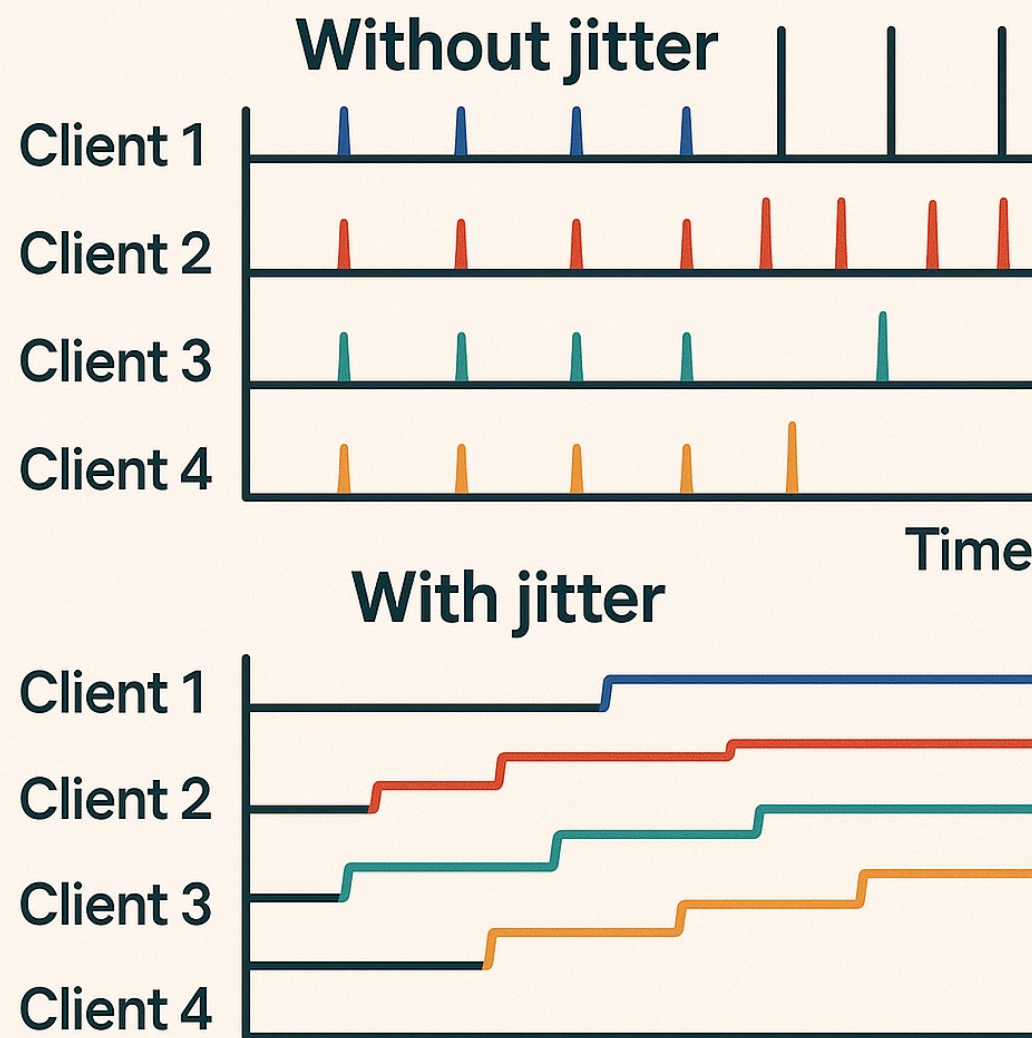
Backoff & jitter (why and how)



Use **exponential backoff** with **jitter** to avoid thundering herds and synchronized retries. Cap the max delay.



Client-side **adaptive throttling** can stabilize systems under overload.



Thundering Herds

When many clients retry failed requests at the same time (e.g., after a transient outage or rate-limit error), they can all flood the server simultaneously.

This synchronized retry surge can **worsen overload** and **delay recovery**.

To solve this:

Increase the delay **exponentially** after each retry attempt:

Example: 1s → 2s → 4s → 8s → 16s ...

This allows transient issues to resolve before retrying, reducing wasted retries.

Always **cap the maximum delay** (e.g., 30 seconds or 1 minute) to prevent excessive waiting.

Adding Jitter (Randomization)

Add a **random factor** ("jitter") to break retry synchronization:

Without jitter, all clients follow the same delay pattern — they still collide.

With jitter, delays are randomized across clients, smoothing retry load.

Common strategies:

Full jitter: wait for a random value between 0 and the current exponential backoff delay.

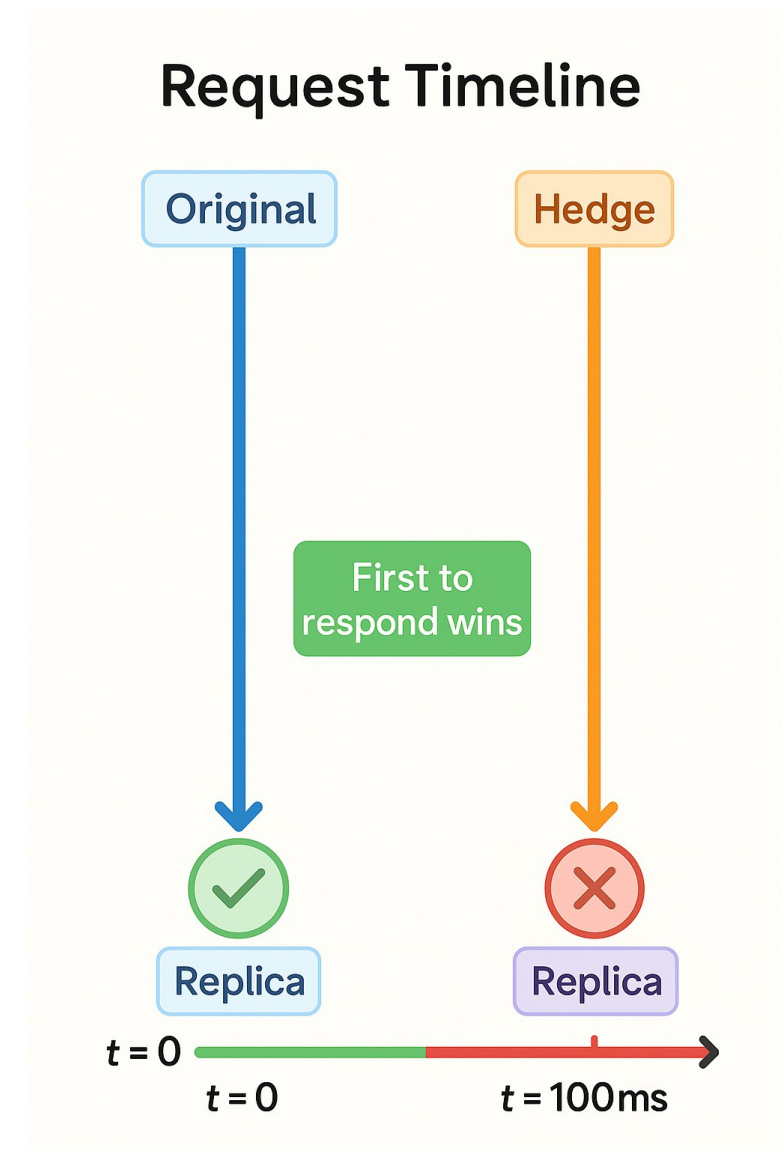
Equal jitter: wait for half of the backoff delay + random half.

Decorrelated jitter: randomize around a moving average of delay times to adapt faster.

Implementing retries — manual vs library

Hedged Requests (Speculative Retries)

- In large systems a small fraction of requests suffer **tail latency** (e.g., p95/p99). Hedging sends a **second copy** of the same request after a short delay; you use the first successful response and cancel the rest.
- This reduces visible tail latency with limited extra load when tuned (delay, max attempts).
- gRPC supports **hedgingPolicy** natively.



When to hedge (and when not)

Good for **read-only or idempotent** operations; dangerous for non-idempotent writes unless guarded by idempotency keys.

Tune **hedgingDelay** and **maxAttempts**; ensure **overall deadline** covers the chain. (gRPC behavior).

Firestore & Firebase in Flutter



Integrated BaaS: Auth, Cloud Firestore (NoSQL), Storage, Cloud Functions, FCM, Hosting, analytics & crash reporting.



Pricing plans: **Spark (free)** and **Blaze (pay-as-you-go)**; cost driven by reads/writes/storage/egress/functions.



Firestore & FCM notes



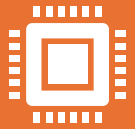
Firestore: client SDKs with realtime listeners and offline cache; good fit for mobile sync.



FCM (push): on iOS, you **must** request notification permission and have APNs token ready before some calls; add required Info.plist keys.



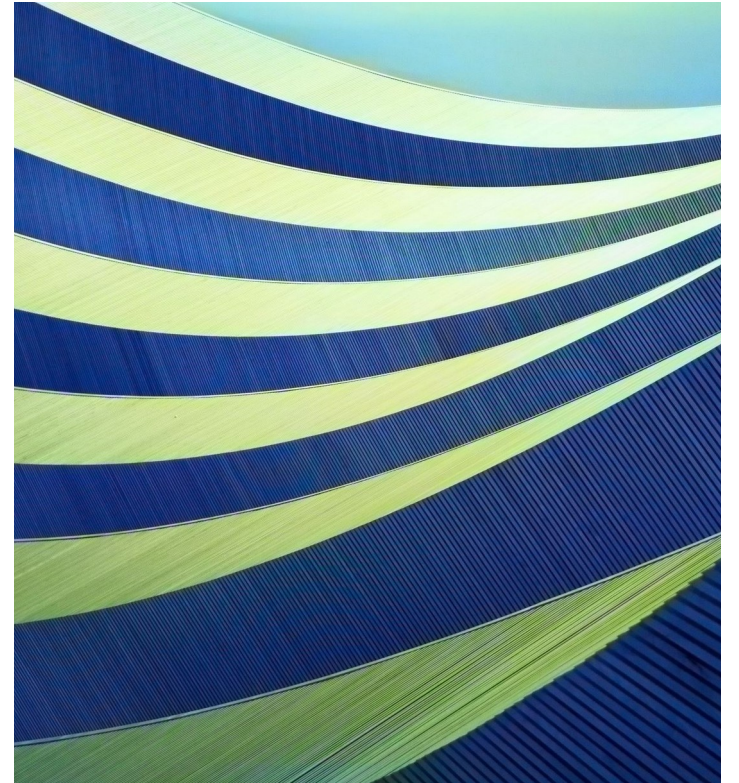
Firestore Functions (serverless)



Trigger on HTTP or events; can reduce cold starts by setting **min instances** if latency-sensitive.



For DB access, reuse clients in module/global scope to benefit from warm instances.



Dart Code Generation

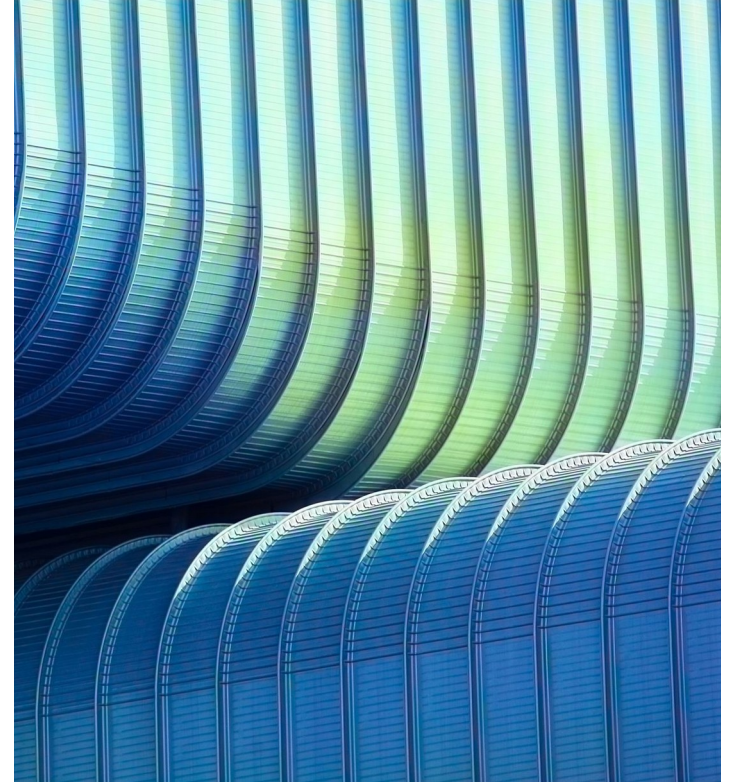
Dart Code Generation



Eliminate boilerplate (JSON models, data classes, clients).



Common approach uses **build_runner** with **source_gen**-based packages.



Core tooling

- **build_runner**: orchestrates builders, writes generated files.

Key commands:

```
dart run build_runner build --  
delete-conflicting-outputs
```

```
dart run build_runner watch
```

- **source_gen**: APIs to write generators over your annotated code.



Popular generators



json_serializable (+
json_annotation): @JsonSerializable() generates toJson/from
Json.



frozen (+ frozen_annotation): immutable classes, unions,
copyWith, deep equality.



retrofit.dart / **chopper**: declarative REST clients via
annotations.

